Server Creation

Section: 6 — HTTP Module and Basic Servers

Subtopic: 02 — Server Creation

Estimated Duration: 3–4 hours

Theory Practical

Topics covered: Listening on ports, `server.listen` options, URL parsing (legacy and WHATWG), querystring parsing, response writing with `res.write`, `res.end`, and `res.setHeader`, manual routing

Learning Objectives — This Subtopic

- Configure a server to listen on specific ports and hosts, and handle the `'listening'` event
- Parse incoming request URLs into their component parts using both the legacy `url` module and the modern `URL` class
- Extract query string parameters from URLs using `querystring` and `URLSearchParams`
- Construct HTTP responses with appropriate headers, status codes, and body content
- Implement basic manual routing by inspecting the request path and method

1. Listening on Ports

In Section 6-1, you created a basic server with `http.createServer()` and called `server.listen(port)`. This section examines `server.listen()` in more detail — its options, its callback, and how port binding actually works.

Create a project folder for this section:

```
mkdir server-creation && cd server-creation
```

1.1 The `listen()` Method

`server.listen()` tells the operating system to start accepting TCP connections on a specific port. The method accepts several argument patterns:

listen-basics.js

```
const http = require('http');

const server = http.createServer((req, res) => {
    res.end('Server is running\n');
});

// listen(port, hostname, backlog, callback)
server.listen(3000, '127.0.0.1', () => {
    const addr = server.address();
    console.log(`Server listening on http://${addr.address}:${addr.port}`);
});
```

```
node listen-basics.js
```

> **Output:**
>
> ```
> Server listening on http://127.0.0.1:3000
> ```

Press `Ctrl` + `C` to stop the server.

What is happening? The server binds to port 3000 on the loopback address `127.0.0.1`. The callback fires once the port is successfully bound. `server.address()` returns an object with `address`, `family` (IPv4/IPv6), and `port` properties.

1.2 Hostname Binding

The hostname argument controls *which network interfaces* the server accepts connections on:

Hostname	Behaviour
`'127.0.0.1'`	Accepts connections from this machine only (loopback)

`'0.0.0.0'`	Accepts connections from any network interface (all incoming)
omitted	Defaults to `'0.0.0.0'` (or `'::'` on IPv6 systems)
`'192.168.1.50'`	Binds to a specific network interface (that IP must exist on the machine)

> **Key Insight:** During development, binding to `127.0.0.1` is safer — it prevents other devices on your network from reaching your server. In production, servers typically bind to `0.0.0.0` to accept connections from anywhere.

1.3 Port Selection and Errors

Ports below 1024 are "well-known ports" and require elevated privileges on most operating systems. If a port is already in use, the server will throw an `EADDRINUSE` error:

listen-error.js

```
const http = require('http');

const server = http.createServer((req, res) => {
    res.end('OK\n');
});

const PORT = 3000;

server.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});

server.on('error', (err) => {
    if (err.code === 'EADDRINUSE') {
        console.error(`Port ${PORT} is already in use.`);
        console.error('Stop the other process or choose a different port.');
    } else if (err.code === 'EACCES') {
        console.error(`Port ${PORT} requires elevated privileges.`);
        console.error('Choose a port above 1024 or run with sudo (not recommended).');
    } else {
        console.error('Server error:', err.message);
```

```
    }
    process.exit(1);
});
```

Start the server, then try to start it again in another terminal to see the error:

`node listen-error.js`

> Output:
>
> ```
> Server running on port 3000
> ```

In a **second terminal**:

`node listen-error.js`

> Output:
>
> ```
> Port 3000 is already in use.
> Stop the other process or choose a different port.
> ```

What is happening? The operating system allows only one process to bind to a given port at a time. The `'error'` event on the server object fires when binding fails. Handling `EADDRINUSE` gracefully is essential — this is one of the most common errors during development.

1.4 Listening on Port 0

Passing port `0` tells the OS to assign any available port automatically. This is useful for testing or when you need to start a server without worrying about port conflicts:

`listen-random-port.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
    res.end('Dynamic port server\n');
});

server.listen(0, () => {
    const { port } = server.address();
```

```
    console.log(`Server assigned port: ${port}`);
    console.log(`Visit: http://127.0.0.1:${port}`);
});
```

```
node listen-random-port.js
```

> **Output:**
>
> ```
> Server assigned port: 49832
> Visit: http://127.0.0.1:49832
> ```

The exact port number will vary each time you run it.

---

2. Parsing Request URLs — The Legacy `url` Module

When a request arrives, `req.url` contains the path and query string — for example, `/products?category=books&page=2` . To work with this, you need to parse it into its component parts. Node.js has two APIs for this: the legacy `url` module and the modern WHATWG `URL` class. Understanding both is important because you will encounter the legacy API in existing codebases.

2.1 `url.parse()`

`url-legacy.js`

```
const url = require('url');

const requestUrl = '/products?category=books&page=2&sort=price';

const parsed = url.parse(requestUrl, true);

console.log('href:    ', parsed.href);
console.log('pathname:', parsed.pathname);
console.log('search:  ', parsed.search);
console.log('query:   ', parsed.query);
console.log('');
```

```
console.log('category:', parsed.query.category);
console.log('page:    ', parsed.query.page);
console.log('sort:    ', parsed.query.sort);
```

`node url-legacy.js`

> Output:
>
> ```
> href:     /products?category=books&page=2&sort=price
> pathname: /products
> search:   ?category=books&page=2&sort=price
> query:    [Object: null prototype] { category: 'books', page: '2', sort: 'price' }
>
> category: books
> page:     2
> sort:     price
> ```

What is happening? `url.parse(urlString, true)` takes a URL string and returns an object with its parts. The second argument (`true`) tells it to parse the query string into an object using the `querystring` module internally. Without `true` , the `query` property is the raw string `'category=books&page=2&sort=price'` .

Key properties of the parsed object:

Property	Value for `/products?category=books&page=2`
`pathname`	`/products`
`search`	`?category=books&page=2`
`query`	`{ category: 'books', page: '2' }` (when `true` is passed)

2.2 The `querystring` Module

The `querystring` module provides dedicated methods for parsing and formatting query strings. While `url.parse()` can handle this automatically (with the second argument), `querystring` is useful when you have a raw query string from another source:

`querystring-demo.js`

```
const querystring = require('querystring');

// Parsing a query string into an object
const qs = 'name=Jane+Doe&age=30&skills=node&skills=express';
const parsed = querystring.parse(qs);

console.log('Parsed:', parsed);
console.log('Name:  ', parsed.name);
console.log('Skills:', parsed.skills);   // Array when key appears multiple times

console.log('---');

// Formatting an object back into a query string
const params = { search: 'hello world', page: 3, limit: 20 };
const formatted = querystring.stringify(params);

console.log('Formatted:', formatted);
```

`node querystring-demo.js`

Output:

```
Parsed: [Object: null prototype] {
  name: 'Jane Doe',
  age: '30',
  skills: [ 'node', 'express' ]
}
Name:   Jane Doe
Skills: [ 'node', 'express' ]
---
Formatted: search=hello%20world&page=3&limit=20
```

What is happening? `querystring.parse()` splits on `&` and `=`, decodes percent-encoding and `+` (as spaces), and returns a plain object. When the same key appears more than once, the value becomes an array. `querystring.stringify()` does the reverse — converts an object into a URL-encoded query string.

> **Note:** All query string values are **strings**. `parsed.age` is `'30'` (string), not `30` (number). Always convert to the expected type with `parseInt()`, `Number()`, or similar when needed.

---

3. Parsing Request URLs — The WHATWG `URL` Class

The legacy `url.parse()` method is still functional but is considered deprecated in favour of the WHATWG `URL` standard. The `URL` class is a global in Node.js (no `require()` needed) and follows the same API used in browsers.

3.1 Basic Usage

`url-modern.js`

```
// The URL class requires a full URL, not just a path.
// Since req.url is only the path, we construct a full URL with a base.
const reqUrl = '/products?category=books&page=2&sort=price';
const base = 'http://localhost:3000';

const parsed = new URL(reqUrl, base);

console.log('href:       ', parsed.href);
console.log('origin:     ', parsed.origin);
console.log('pathname:   ', parsed.pathname);
console.log('search:     ', parsed.search);
console.log('searchParams:', parsed.searchParams);

console.log('');
console.log('category:', parsed.searchParams.get('category'));
console.log('page:    ', parsed.searchParams.get('page'));
console.log('sort:    ', parsed.searchParams.get('sort'));
console.log('missing: ', parsed.searchParams.get('missing'));
```

`node url-modern.js`

Output:

```
href:       http://localhost:3000/products?category=books&page=2&sort=price
origin:     http://localhost:3000
pathname:   /products
search:     ?category=books&page=2&sort=price
searchParams: URLSearchParams { 'category' => 'books', 'page' => '2', 'sort' => 'price' }

category: books
page:     2
sort:     price
missing:  null
```

What is happening? The `URL` constructor takes two arguments: the URL string and an optional base. Since `req.url` in an HTTP server is a relative path (e.g., `/products?page=2`), you need to provide a base URL. The base does not need to be the actual server address — it is only used to construct a valid absolute URL for parsing.

3.2 `URLSearchParams`

The `searchParams` property of a `URL` object is a `URLSearchParams` instance, which provides a richer API than the plain object returned by the legacy module:

`search-params.js`

```
const reqUrl = '/search?tag=node&tag=javascript&tag=express&limit=10';
const parsed = new URL(reqUrl, 'http://localhost');
const params = parsed.searchParams;

// get() returns the FIRST value for a key
console.log('First tag:', params.get('tag'));

// getAll() returns ALL values for a key as an array
console.log('All tags: ', params.getAll('tag'));

// has() checks if a key exists
console.log('Has limit:', params.has('limit'));
console.log('Has page: ', params.has('page'));

// Iterate over all entries
console.log('\nAll parameters:');
for (const [key, value] of params) {
    console.log(`  ${key} = ${value}`);
}

// Convert to a plain object (loses duplicate keys)
const plain = Object.fromEntries(params);
console.log('\nAs plain object:', plain);
```

`node search-params.js`

Output:

```
First tag: node
All tags:  [ 'node', 'javascript', 'express' ]
Has limit: true
```

```
Has page:  false

All parameters:
  tag = node
  tag = javascript
  tag = express
  limit = 10

As plain object: { tag: 'express', limit: '10' }
```

What is happening? `URLSearchParams` handles duplicate keys properly through `getAll()`. It is iterable, so you can use `for...of` loops. The `Object.fromEntries()` conversion loses duplicate keys (only the last value survives), so avoid it when duplicate parameters are expected.

3.3 Legacy vs Modern — Which to Use

Feature	Legacy `url.parse()`	WHATWG `URL`
Requires `require()`	Yes — `require('url')`	No — global
Handles relative paths	Yes — directly	Needs a base URL
Query parsing	Returns plain object	Returns `URLSearchParams` with richer API
Duplicate query keys	Last value wins (or array with `querystring`)	`getAll()` returns all values
Status	Legacy — still works, but not recommended for new code	Standard — recommended approach

> **Key Insight:** For new code, prefer the WHATWG `URL` class. You will encounter `url.parse()` in older projects and documentation, so knowing both is valuable. The pattern `new URL(req.url, 'http://localhost')` is a common idiom you will see frequently.

---

4. Writing Responses

In Section 6-1, you saw `res.end('Hello')` to send a simple response. This section examines the full response API: setting headers, writing the body in parts, and sending structured data.

4.1 `res.setHeader()` and `res.writeHead()`

response-headers.js

```
const http = require('http');

const server = http.createServer((req, res) => {
    // Method 1: Set headers individually
    res.setHeader('Content-Type', 'text/plain');
    res.setHeader('X-Powered-By', 'Node.js');
    res.setHeader('Cache-Control', 'no-cache');

    // Read back a header
    console.log('Content-Type:', res.getHeader('Content-Type'));

    // Remove a header
    res.removeHeader('X-Powered-By');

    res.statusCode = 200;
    res.end('Headers set individually\n');
});

server.listen(3000, () => {
    console.log('Server running on http://127.0.0.1:3000');
    console.log('Test with: curl -i http://127.0.0.1:3000');
});
```

```
node response-headers.js
```

In another terminal:

```
curl -i http://127.0.0.1:3000
```

> Output:

```
HTTP/1.1 200 OK
Content-Type: text/plain
Cache-Control: no-cache
Date: Wed, 15 Jan 2025 14:30:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

Headers set individually
```

What is happening? `res.setHeader(name, value)` sets a single header. `res.statusCode` sets the HTTP status. Headers and status are sent when you first call `res.write()` or `res.end()`. Notice that `X-Powered-By` is absent — we removed it with `res.removeHeader()`.

There is also `res.writeHead()`, which sets the status code and all headers in a single call:

`response-writehead.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
    // Method 2: Set status + headers in one call
    res.writeHead(200, {
        'Content-Type': 'application/json',
        'X-Request-Id': '12345'
    });

    const data = { message: 'Headers set with writeHead', timestamp: Date.now() };
    res.end(JSON.stringify(data));
});

server.listen(3001, () => {
    console.log('Server running on http://127.0.0.1:3001');
});
```

```
node response-writehead.js
```

```
curl -i http://127.0.0.1:3001
```

Output:

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Request-Id: 12345
```

```
Date: Wed, 15 Jan 2025 14:31:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"message":"Headers set with writeHead","timestamp":1736951460000}
```

> **Important:** Once headers have been sent (after the first `res.write()` or `res.end()`), you cannot modify them. Calling `res.setHeader()` after headers are sent throws an `ERR_HTTP_HEADERS_SENT` error. Always set all headers before writing the body.

4.2 `res.write()` and `res.end()`

`res.write()` sends a chunk of the response body without closing the connection. `res.end()` sends the final chunk (if any) and signals that the response is complete. This allows you to build the response incrementally:

response-write.js

```
const http = require('http');

const server = http.createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'text/html' });

    // Build the response in parts
    res.write('<!DOCTYPE html>');
    res.write('<html><head><title>Chunked Response</title></head>');
    res.write('<body>');
    res.write('<h1>Built with res.write()</h1>');
    res.write('<p>This response was sent in multiple chunks.</p>');
    res.write('<p>Useful for large responses or streamed data.</p>');
    res.write('</body></html>');

    // Finish the response
    res.end();
});

server.listen(3000, () => {
    console.log('Server running on http://127.0.0.1:3000');
});
```

```
node response-write.js
```

Open `http://127.0.0.1:3000` in your browser to see the rendered HTML.

> **Output:**
>
> ```
> (Browser renders:)
> Built with res.write()
> This response was sent in multiple chunks.
> Useful for large responses or streamed data.
> ```

What is happening? Each `res.write()` call sends data to the client. The client (browser) may start rendering before the full response arrives. `res.end()` with no arguments closes the response. You can also pass a final chunk to `res.end(data)` as a shorthand for `res.write(data)` followed by `res.end()`.

> **Warning:** Every request handler **must** call `res.end()`. If you forget, the client will hang indefinitely waiting for the response to complete, and the connection will eventually time out. This is a very common mistake.

4.3 Sending JSON Responses

Sending JSON is one of the most common operations in a server. The pattern is always the same: set the `Content-Type` header, serialise the data with `JSON.stringify()`, and send it:

response-json.js

```
const http = require('http');

function sendJSON(res, statusCode, data) {
    const body = JSON.stringify(data);

    res.writeHead(statusCode, {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(body)
    });

    res.end(body);
}

const server = http.createServer((req, res) => {
```

```
    sendJSON(res, 200, {
        status: 'success',
        data: { id: 1, name: 'Introduction to Node.js', duration: '8 hours' }
    });
});

server.listen(3000, () => {
    console.log('JSON server on http://127.0.0.1:3000');
});
```

`node response-json.js`

`curl http://127.0.0.1:3000`

> Output:
>
> `{"status":"success","data":{"id":1,"name":"Introduction to Node.js","duration":"8 hours"}}`

What is happening? We wrapped the JSON response logic in a reusable `sendJSON()` helper function. Setting `Content-Length` with `Buffer.byteLength(body)` is a best practice — it tells the client exactly how many bytes to expect. Note that we use `Buffer.byteLength()` rather than `body.length` because multi-byte characters (such as accented letters) take more bytes than their string length suggests.

---

5. Manual Routing

With the ability to parse URLs and write responses, you can now implement routing — directing each request to the correct handler based on its path and HTTP method.

> **Analogy:** Think of manual routing like a receptionist at a building's front desk. Every visitor (request) arrives at the same entrance (the server). The receptionist checks who they want to see (the URL path) and what they need (the HTTP method), then directs them to the correct office. If no office matches, the receptionist says "that department does not exist" (404).

5.1 A Basic Router

Stage 1 **Routing by path only** `router-basic.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
    const { pathname } = new URL(req.url, 'http://localhost');

    res.setHeader('Content-Type', 'application/json');

    if (pathname === '/') {
        res.statusCode = 200;
        res.end(JSON.stringify({ message: 'Welcome to the API' }));

    } else if (pathname === '/health') {
        res.statusCode = 200;
        res.end(JSON.stringify({ status: 'healthy', uptime: process.uptime() }));

    } else if (pathname === '/time') {
        res.statusCode = 200;
        res.end(JSON.stringify({ time: new Date().toISOString() }));

    } else {
        res.statusCode = 404;
        res.end(JSON.stringify({ error: 'Not Found', path: pathname }));
    }
});

server.listen(3000, () => {
    console.log('Router running on http://127.0.0.1:3000');
});
```

```
node router-basic.js
```

Test each route:

```
curl http://127.0.0.1:3000/
```

Output:

```
{"message":"Welcome to the API"}
```

```
curl http://127.0.0.1:3000/health
```

Output:

```
{"status":"healthy","uptime":4.567}
```

```
curl http://127.0.0.1:3000/unknown
```

Output:

```
{"error":"Not Found","path":"/unknown"}
```

What is happening? We use `new URL(req.url, 'http://localhost').pathname` to extract the clean path without query parameters. An `if/else` chain maps each path to its handler. Any unrecognised path falls through to the 404 response.

5.2 Routing by Path and Method

Stage 2 **Adding HTTP method awareness** `router-methods.js`

```
const http = require('http');

// In-memory data store
const books = [
    { id: 1, title: 'The Pragmatic Programmer', author: 'Hunt & Thomas' },
    { id: 2, title: 'Clean Code', author: 'Robert C. Martin' },
    { id: 3, title: 'Node.js Design Patterns', author: 'Mario Casciaro' }
];

function sendJSON(res, statusCode, data) {
    const body = JSON.stringify(data, null, 2);
    res.writeHead(statusCode, {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(body)
    });
    res.end(body);
}

const server = http.createServer((req, res) => {
    const { pathname, searchParams } = new URL(req.url, 'http://localhost');
    const method = req.method;
```

```
    // GET /books — list all books (with optional author filter)
    if (pathname === '/books' && method === 'GET') {
        const authorFilter = searchParams.get('author');

        let results = books;
        if (authorFilter) {
            results = books.filter(b =>
                b.author.toLowerCase().includes(authorFilter.toLowerCase())
            );
        }

        sendJSON(res, 200, { count: results.length, books: results });

    // GET /books/:id — get a single book
    } else if (pathname.startsWith('/books/') && method === 'GET') {
        const id = parseInt(pathname.split('/')[2], 10);
        const book = books.find(b => b.id === id);

        if (book) {
            sendJSON(res, 200, book);
        } else {
            sendJSON(res, 404, { error: `Book with id ${id} not found` });
        }

    // GET / — root
    } else if (pathname === '/' && method === 'GET') {
        sendJSON(res, 200, {
            message: 'Book API',
            endpoints: ['GET /books', 'GET /books/:id', 'GET /books?author=name']
        });

    // Method not allowed
    } else if (['/books', '/'].includes(pathname)) {
        res.writeHead(405, {
            'Content-Type': 'application/json',
            'Allow': 'GET'
        });
        res.end(JSON.stringify({ error: 'Method Not Allowed' }));

    // Not found
    } else {
        sendJSON(res, 404, { error: 'Not Found' });
    }
});
```

```
server.listen(3000, () => {
    console.log('Book API on http://127.0.0.1:3000');
});
```

`node router-methods.js`

Test the various routes:

`curl http://127.0.0.1:3000/`

Output:

```
{
  "message": "Book API",
  "endpoints": [
    "GET /books",
    "GET /books/:id",
    "GET /books?author=name"
  ]
}
```

`curl http://127.0.0.1:3000/books`

Output:

```
{
  "count": 3,
  "books": [
    { "id": 1, "title": "The Pragmatic Programmer", "author": "Hunt & Thomas" },
    { "id": 2, "title": "Clean Code", "author": "Robert C. Martin" },
    { "id": 3, "title": "Node.js Design Patterns", "author": "Mario Casciaro" }
  ]
}
```

`curl http://127.0.0.1:3000/books?author=martin`

Output:

```
{
  "count": 1,
  "books": [
```

```
    { "id": 2, "title": "Clean Code", "author": "Robert C. Martin" }
  ]
}
```

```
curl http://127.0.0.1:3000/books/3
```

Output:

```
{
  "id": 3,
  "title": "Node.js Design Patterns",
  "author": "Mario Casciaro"
}
```

```
curl http://127.0.0.1:3000/books/99
```

Output:

```
{
  "error": "Book with id 99 not found"
}
```

```
curl -X DELETE http://127.0.0.1:3000/books
```

Output:

```
{
  "error": "Method Not Allowed"
}
```

What is happening? The router now checks both `pathname` and `req.method`. We extract the `searchParams` from the parsed URL to handle query filtering (`?author=martin`). For paths like `/books/3`, we extract the ID by splitting the path on `/`. The 405 (Method Not Allowed) response includes an `Allow` header listing the permitted methods — this is required by the HTTP specification.

> **Key Insight:** This manual routing pattern — `if/else` chains checking path and method — works for small servers but becomes unmanageable as the number of routes grows. This is exactly the problem that Express (Section 7) solves with its `app.get()`, `app.post()` routing API. Building it manually first gives you a clear understanding of what Express does under the hood.

5.3 Extracting Path Parameters

The pattern of extracting an ID from `/books/3` using `pathname.split('/')[2]` works for simple cases, but let us make it more robust with a small helper:

`path-params.js`

```
const http = require('http');

/**
 * Match a URL path against a pattern with named parameters.
 * Pattern: '/books/:id/reviews/:reviewId'
 * Path:    '/books/5/reviews/42'
 * Returns: { id: '5', reviewId: '42' } or null if no match
 */
function matchRoute(pattern, path) {
    const patternParts = pattern.split('/');
    const pathParts = path.split('/');

    if (patternParts.length !== pathParts.length) return null;

    const params = {};

    for (let i = 0; i < patternParts.length; i++) {
        if (patternParts[i].startsWith(':')) {
            // This is a parameter — capture it
            const paramName = patternParts[i].slice(1);
            params[paramName] = pathParts[i];
        } else if (patternParts[i] !== pathParts[i]) {
            // Static segment does not match
            return null;
        }
    }

    return params;
}

// --- Test the helper ---
```

```
console.log(matchRoute('/books/:id', '/books/5'));
console.log(matchRoute('/books/:id', '/books'));
console.log(matchRoute('/books/:id/reviews/:reviewId', '/books/5/reviews/42'));
console.log(matchRoute('/books/:id', '/authors/5'));
```

`node path-params.js`

Output:

```
{ id: '5' }
null
{ id: '5', reviewId: '42' }
null
```

What is happening? The `matchRoute()` function compares each segment of the URL pattern with the actual path. Segments starting with `:` are treated as named parameters and their values are captured into an object. If any static segment does not match, or the number of segments differs, it returns `null`. This is a simplified version of the route matching that Express performs internally.

---

6. Putting It All Together

Let us combine everything from this section into a single server that demonstrates URL parsing, query parameters, multiple routes, proper headers, and structured JSON responses:

`combined-server.js`

```
const http = require('http');

// --- In-memory data ---
const courses = [
    { id: 1, title: 'Node.js Fundamentals', category: 'backend', hours: 8 },
    { id: 2, title: 'Express Framework', category: 'backend', hours: 12 },
    { id: 3, title: 'React Basics', category: 'frontend', hours: 10 },
    { id: 4, title: 'CSS Grid & Flexbox', category: 'frontend', hours: 6 }
];

// --- Helpers ---
function sendJSON(res, statusCode, data) {
    const body = JSON.stringify(data, null, 2);
```

```
    res.writeHead(statusCode, {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(body)
    });
    res.end(body);
}

function send404(res) {
    sendJSON(res, 404, { error: 'Not Found' });
}

// --- Server ---
const server = http.createServer((req, res) => {
    const parsed = new URL(req.url, 'http://localhost');
    const path = parsed.pathname;
    const method = req.method;
    const params = parsed.searchParams;

    console.log(`${method} ${req.url}`);

    // --- Routes ---

    // GET / — API overview
    if (path === '/' && method === 'GET') {
        sendJSON(res, 200, {
            name: 'Course Catalogue API',
            version: '1.0.0',
            endpoints: [
                'GET  /courses               — List all courses',
                'GET  /courses?category=x     — Filter by category',
                'GET  /courses?minHours=n     — Filter by minimum hours',
                'GET  /courses/:id            — Get a single course',
                'GET  /courses/:id/summary    — Get a brief summary'
            ]
        });
        return;
    }

    // GET /courses/:id/summary
    if (path.match(/^\/courses\/\d+\/summary$/) && method === 'GET') {
        const id = parseInt(path.split('/')[2], 10);
        const course = courses.find(c => c.id === id);

        if (!course) return sendJSON(res, 404, { error: `Course ${id} not found` });

        // Return a text summary instead of JSON
        res.writeHead(200, { 'Content-Type': 'text/plain' });
```

```
        res.end(`${course.title} (${course.category}) — ${course.hours} hours`);
        return;
    }

    // GET /courses/:id
    if (path.match(/^\/courses\/\d+$/) && method === 'GET') {
        const id = parseInt(path.split('/')[2], 10);
        const course = courses.find(c => c.id === id);

        if (!course) return sendJSON(res, 404, { error: `Course ${id} not found` });

        sendJSON(res, 200, course);
        return;
    }

    // GET /courses
    if (path === '/courses' && method === 'GET') {
        let results = [...courses];

        // Filter by category
        const category = params.get('category');
        if (category) {
            results = results.filter(c => c.category === category.toLowerCase());
        }

        // Filter by minimum hours
        const minHours = params.get('minHours');
        if (minHours) {
            const min = parseInt(minHours, 10);
            if (!isNaN(min)) {
                results = results.filter(c => c.hours >= min);
            }
        }

        sendJSON(res, 200, { count: results.length, courses: results });
        return;
    }

    // Fallback — 404
    send404(res);
});

const PORT = 3000;

server.on('error', (err) => {
    if (err.code === 'EADDRINUSE') {
        console.error(`Port ${PORT} is already in use.`);
```

```
    } else {
        console.error('Server error:', err.message);
    }
    process.exit(1);
});

server.listen(PORT, () => {
    console.log(`Course API running on http://127.0.0.1:${PORT}`);
});
```

```
node combined-server.js
```

Test the filtering:

```
curl "http://127.0.0.1:3000/courses?category=frontend"
```

Output:

```
{
  "count": 2,
  "courses": [
    { "id": 3, "title": "React Basics", "category": "frontend", "hours": 10 },
    { "id": 4, "title": "CSS Grid & Flexbox", "category": "frontend", "hours": 6 }
  ]
}
```

```
curl "http://127.0.0.1:3000/courses?minHours=10"
```

Output:

```
{
  "count": 2,
  "courses": [
    { "id": 2, "title": "Express Framework", "category": "backend", "hours": 12 },
    { "id": 3, "title": "React Basics", "category": "frontend", "hours": 10 }
  ]
}
```

```
curl http://127.0.0.1:3000/courses/2/summary
```

Output:

```
Express Framework (backend) — 12 hours
```

What is happening? This server demonstrates several patterns working together: the WHATWG `URL` class for parsing, `searchParams` for query filters, regex-based path matching for parameterised routes, different content types for different endpoints (`application/json` vs `text/plain`), and proper error handling on the server object. The request logging (`console.log`) in the handler shows each incoming request in the terminal — a basic form of request logging that you will formalise with middleware in Section 7.

Summary

Concept	Key Point
`server.listen()`	Binds to a port and host; callback fires when ready; port 0 for auto-assignment
Hostname binding	`127.0.0.1` for local only, `0.0.0.0` for all interfaces; omitted defaults to all
`EADDRINUSE`	Port already in use — handle via `server.on('error')`
`url.parse()` (legacy)	Parses URL string; pass `true` to parse query into object; requires `require('url')`
`querystring` module	`parse()` and `stringify()` for query strings; handles duplicate keys as arrays
`new URL()` (WHATWG)	Modern standard; needs a base URL for relative paths; global — no require needed
`URLSearchParams`	`get()` , `getAll()` , `has()` , iterable; preferred over legacy `querystring`
`res.setHeader()`	Sets individual headers before sending; `res.writeHead()` sets status + headers at once

`res.write()` / `res.end()`	`write()` sends partial body; `end()` sends final chunk and closes response
Manual routing	Check `pathname` + `req.method`; works for small servers but does not scale — Express solves this

---

Interview Questions

1. What does `server.listen(0)` do, and when would you use it?
Expected answer: Passing port 0 tells the operating system to assign any available port. This is useful in automated testing where multiple test suites may run in parallel and you need to avoid port conflicts. After the server starts, call `server.address().port` to find out which port was assigned.

2. What is the difference between `res.setHeader()` and `res.writeHead()`?
Expected answer: `res.setHeader(name, value)` sets individual headers and can be called multiple times before sending. `res.writeHead(statusCode, headers)` sets the status code and all headers in one call and immediately queues them for sending. Headers set with `setHeader()` are merged with those in `writeHead()`, but `writeHead()` takes precedence for duplicate headers.

3. Why should you use `Buffer.byteLength(body)` instead of `body.length` for the `Content-Length` header?
Expected answer: `String.length` counts characters, but `Content-Length` must be in bytes. Multi-byte characters (e.g., emoji, accented characters, non-Latin scripts) use more bytes than their character count. `Buffer.byteLength()` returns the actual byte count for the given encoding.

4. What happens if you call `res.setHeader()` after `res.write()` has already been called?
Expected answer: Node.js throws an `ERR_HTTP_HEADERS_SENT` error. Once `res.write()` or `res.end()` is called, headers are flushed to the client and cannot be modified. All headers must be set before any body content is sent.

5. Explain the difference between `url.parse()` and `new URL()`.
Expected answer: `url.parse()` is the legacy Node.js API — it requires `require('url')`, accepts relative paths directly, and returns a plain object with a `query` property. `new URL()` is the

WHATWG standard — it is a global, requires a base URL for relative paths, and provides `searchParams` (a `URLSearchParams` object) with methods like `get()`, `getAll()`, and `has()`. The modern API is the recommended approach.

6. What HTTP status code should you return when a client sends a valid path but uses the wrong method (e.g., DELETE on a read-only endpoint)?
Expected answer: 405 Method Not Allowed. The response should include an `Allow` header listing the permitted methods for that path (e.g., `Allow: GET`). This is distinct from 404 Not Found (path does not exist) and 400 Bad Request (malformed request).

7. Why does every request handler need to call `res.end()`?
Expected answer: Without `res.end()`, the server never signals that the response is complete. The client will hang waiting for more data until the connection times out. In the meantime, the connection remains open, consuming resources on both the server and the client.

8. What is the limitation of manual routing with `if/else` chains, and what does Express provide as an alternative?
Expected answer: Manual routing becomes unmanageable as routes grow — deeply nested conditionals, duplicated logic, no easy way to share cross-cutting concerns (logging, authentication). Express provides a declarative routing API (`app.get()`, `app.post()`), a middleware system for shared logic, the Router class for modular route organisation, and built-in parameter extraction (`req.params`).

Practical Assignment — Product Catalogue Mini API

Objective: Build a read-only Product Catalogue API using only the core `http` module. The API serves product data from an in-memory array, supports filtering via query parameters, and returns appropriate status codes and headers for all routes.

Step 1: Create a project folder:

```
mkdir product-api && cd product-api
```

Step 2: Create the data file:

`data.js`

```
const products = [
    { id: 1, name: 'Mechanical Keyboard', category: 'peripherals', price: 129.99,
inStock: true },
    { id: 2, name: '27" 4K Monitor', category: 'displays', price: 449.00, inStock:
true },
    { id: 3, name: 'USB-C Hub', category: 'peripherals', price: 59.99, inStock: false },
    { id: 4, name: 'Wireless Mouse', category: 'peripherals', price: 79.99, inStock:
true },
    { id: 5, name: 'Standing Desk', category: 'furniture', price: 599.00, inStock:
true },
    { id: 6, name: 'Webcam 1080p', category: 'peripherals', price: 89.99, inStock:
false },
    { id: 7, name: 'Ergonomic Chair', category: 'furniture', price: 899.00, inStock:
true },
    { id: 8, name: 'Portable SSD 1TB', category: 'storage', price: 109.99, inStock:
true }
];

module.exports = products;
```

Step 3: Create the server with the following routes:

Method	Path	Description
GET	`/`	API information and available endpoints
GET	`/products`	List all products
GET	`/products?category=x`	Filter by category
GET	`/products?inStock=true`	Filter by stock status
GET	`/products?maxPrice=n`	Filter by maximum price
GET	`/products/:id`	Get a single product by ID
GET	`/categories`	List all unique categories

Requirements:

- All query filters should be combinable (e.g., `?category=peripherals&inStock=true`)
- Return 404 with a JSON error body for unknown routes
- Return 405 with an `Allow` header for non-GET requests to valid routes
- Set `Content-Type` and `Content-Length` on all responses
- Handle `EADDRINUSE` gracefully
- Use the WHATWG `URL` class for URL parsing

`server.js`

```
const http = require('http');
const products = require('./data');

function sendJSON(res, statusCode, data) {
    const body = JSON.stringify(data, null, 2);
    res.writeHead(statusCode, {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(body)
    });
    res.end(body);
}

const server = http.createServer((req, res) => {
    const parsed = new URL(req.url, 'http://localhost');
    const path = parsed.pathname;
    const method = req.method;
    const params = parsed.searchParams;

    // Log every request
    console.log(`${new Date().toISOString()} ${method} ${req.url}`);

    // Only allow GET requests
    if (method !== 'GET') {
        res.writeHead(405, {
            'Content-Type': 'application/json',
            'Allow': 'GET',
            'Content-Length': Buffer.byteLength(JSON.stringify({ error: 'Method Not
Allowed' }))
        });
        res.end(JSON.stringify({ error: 'Method Not Allowed' }));
        return;
    }
```

```
// GET /
if (path === '/') {
    sendJSON(res, 200, {
        name: 'Product Catalogue API',
        version: '1.0.0',
        endpoints: [
            'GET /products',
            'GET /products?category=x&inStock=true&maxPrice=n',
            'GET /products/:id',
            'GET /categories'
        ]
    });
    return;
}

// GET /categories
if (path === '/categories') {
    const categories = [...new Set(products.map(p => p.category))].sort();
    sendJSON(res, 200, { categories });
    return;
}

// GET /products/:id
if (path.match(/^\/products\/\d+$/)) {
    const id = parseInt(path.split('/')[2], 10);
    const product = products.find(p => p.id === id);

    if (!product) {
        sendJSON(res, 404, { error: `Product with id ${id} not found` });
    } else {
        sendJSON(res, 200, product);
    }
    return;
}

// GET /products
if (path === '/products') {
    let results = [...products];

    // Filter: category
    const category = params.get('category');
    if (category) {
        results = results.filter(p => p.category === category.toLowerCase());
    }

    // Filter: inStock
```

```
        const inStock = params.get('inStock');
        if (inStock !== null) {
            const stockFilter = inStock === 'true';
            results = results.filter(p => p.inStock === stockFilter);
        }

        // Filter: maxPrice
        const maxPrice = params.get('maxPrice');
        if (maxPrice !== null) {
            const max = parseFloat(maxPrice);
            if (!isNaN(max)) {
                results = results.filter(p => p.price <= max);
            }
        }

        sendJSON(res, 200, { count: results.length, products: results });
        return;
    }

    // 404 — no route matched
    sendJSON(res, 404, { error: 'Not Found', path });
});

const PORT = 3000;

server.on('error', (err) => {
    if (err.code === 'EADDRINUSE') {
        console.error(`Port ${PORT} is already in use. Stop the other process or use a
different port.`);
    } else {
        console.error('Server error:', err.message);
    }
    process.exit(1);
});

server.listen(PORT, () => {
    console.log(`Product Catalogue API running on http://127.0.0.1:${PORT}`);
});
```

Step 4: Run and test:

```
node server.js
```

```
curl "http://127.0.0.1:3000/products?category=peripherals&inStock=true"
```

Expected output:

Output:

```
{
  "count": 2,
  "products": [
    { "id": 1, "name": "Mechanical Keyboard", "category": "peripherals", "price": 129.99, "inStock":
true },
    { "id": 4, "name": "Wireless Mouse", "category": "peripherals", "price": 79.99, "inStock": true }
  ]
}
```

`curl http://127.0.0.1:3000/categories`

Output:

```
{
  "categories": [
    "displays",
    "furniture",
    "peripherals",
    "storage"
  ]
}
```

```
product-api/
├── data.js
└── server.js
```

Bonus Challenge

Add a `GET /products/stats` route that returns aggregate statistics: total product count, average price (rounded to 2 decimal places), number in stock vs out of stock, and a breakdown of products per category. Make sure this route is checked *before* the `/products/:id` route so that `"stats"` is not interpreted as an ID.